Analytics Engineering: Session 11

Model Governance: Contracts, Versions & Access

Agenda

- Why Model Governance?
- Model Contracts
- Model Versions & Deprecation
- Model Access Levels
- Breaking vs Non-breaking Changes

Why Model Governance?

As dbt projects grow, teams need rules to prevent downstream breakage.

- **Contracts**: Guarantee the shape of your data (schema enforcement).
- **Versions**: Allow breaking changes without immediately breaking consumers.
- **Access**: Control who can reference which models.

Governance = Contracts + Versions + Access

Model Contracts

Enforce the shape of your data (schema) at build time.

```yaml
models:
  - name: dim_customers
    config:
      contract: {enforced: true}
    columns:
      - name: customer_id
        data_type: int
        constraints:
          - type: not_null
      - name: email
        data_type: varchar
```

If the SQL returns a different type for `customer_id`, the build **fails**.

Contract Constraints

Available constraints you can enforce:

Constraint	Behavior
`not_null`	Column cannot contain nulls
`unique`	Column values must be unique
`primary_key`	Combination of not_null + unique
`check`	Custom SQL expression must be true

```yaml
columns:
  - name: total_amount
    data_type: numeric
```

Model Versions

Manage breaking changes to your models without disrupting downstream consumers.

```yaml
models:
  - name: dim_customers
    latest_version: 2
    versions:
      - v: 1
        columns:
          - include: all
      - v: 2
        columns:
          - include: all
          - name: loyalty_tier
            data_type: varchar
```


Consumers reference a specific version: `{{ ref('dim_customers', v=1) }}`

Deprecation

Signal that a model version is going away.

```yaml
versions:
  - v: 1
    deprecation_date: 2025-12-31
  - v: 2
```

- dbt will **warn** users referencing v1 during builds.
- After the date, teams should migrate to v2.
- Deprecation is a communication tool, not an enforcement mechanism.

Model Access

Control which models can be referenced by other projects or groups.

Level	Who can `ref()` it?
public	Any project or group
protected	Same project only (default)
private	Same group only

```yaml
models:
  - name: dim_customers
    access: public
  - name: int_order_items_summary
    access: private
```

Breaking vs Non-breaking Changes

Non-breaking (safe to deploy):

- Adding a new column
- Changing a column description
- Relaxing a constraint (not_null → nullable)

Breaking (requires versioning):

- Removing a column
- Renaming a column
- Changing a column's data type
- Changing the grain of the model

When in doubt, version it.

Putting It All Together

Combine Contracts + Versions + Access for a **Data Mesh Ready** project:

```yaml
models:
  - name: dim_customers
    access: public
    config:
      contract: {enforced: true}
    latest_version: 2
    versions:
      - v: 1
        deprecation_date: 2025-12-31
      - v: 2
    columns:
      - name: customer_id
        data_type: int
        constraints:
```

What have we learned in this session

- Add contracts to enforce schema at build time
- Version models to handle breaking changes safely
- Set deprecation dates to communicate migration timelines
- Configure access levels (public, protected, private)
- Distinguish between breaking and non-breaking changes

Next Session: Practice Session III: Production-Ready Data Quality.